TP3 — Perceptrón Simple y Multicapa

Sistemas de Inteligencia Artificial — ITBA 2026

Equipo SIA

Marco teórico y objetivos

Objetivos del TP3

- **Ejercicio 1**: Estudio de fraude online con perceptrón simple lineal vs no lineal (knowledge distillation de *BigModel*).
- **Ejercicio 2**: Clasificación de dígitos con MLP, variando tasa de aprendizaje, arquitectura y mecanismo de optimización.
- **Ejercicio 3**: Mejorar el clasificador con `more_digits.csv` buscando **accuracy** $\geq 98\,\%$.

Esta presentación resume las decisiones más relevantes de **EJ1, EJ2 y EJ3**.

Arquitectura del MLP (Clase 11, Clase 13 - Backprop)

Forward: $V_j^m = \theta\left(\sum_k V_k^{m-1} \cdot w_{jk}^m\right)$ con $V^0 = x$, $O = V^M$.

Loss (MSE): $E(w) = \frac{1}{2N} \sum_{\mu=1}^{N} \sum_i \left(\zeta_i^\mu - O_i^\mu\right)^2$

Backward (regla de la cadena):

$$\delta^{(L)} = (O - t) \odot f'(h^{(L)}), \quad \delta^{(m)} = \left(W^{(m+1)\top} \delta^{(m+1)}\right) \odot f'(h^{(m)})$$

$$\frac{\partial E}{\partial W^{(m)}} = \frac{1}{N} \delta^{(m)} V^{(m-1)\top}, \quad w^{\text{nuevo}} = w^{\text{ant}} - \eta \frac{\partial E}{\partial w}$$

Capacidad y generalización (Clase 13 - Regularización): más capacidad $\Rightarrow$ más riesgo de overfitting; brecha train$-$validación crece con la capacidad.

EJ1 — Prevención de fraude

EJ1 — Contexto y setup

Problema: CompanyX usa un modelo grande y caro, **BigModel**, para estimar la probabilidad de fraude en transacciones online.

Objetivo: construir un modelo más chico, **TinyModel**, que aproxime la salida de BigModel con menor costo de inferencia.

Qué se pide estudiar:

- Comparar un perceptrón simple lineal contra uno no lineal.
- Elegir el modelo con mejor capacidad de aprendizaje para generalización.
- Recomendar un umbral de detección de fraude.

Criterio de negocio: priorizar la reducción de falsos negativos, porque un fraude real no detectado es más costoso que revisar una operación sospechosa.

EJ1 — Normalización con z-score

Usamos **z-score** para llevar las variables a una escala comparable antes del entrenamiento.

$$z = \frac{x - \mu}{\sigma}$$

donde μ es la **media** y σ es el **desvío estándar**.

Ejemplo real sobre `amount_usd` del dataset:

$$x = [42{,}07]\,, \qquad \mu = [109{,}07]\,, \qquad \sigma = [157{,}74]$$

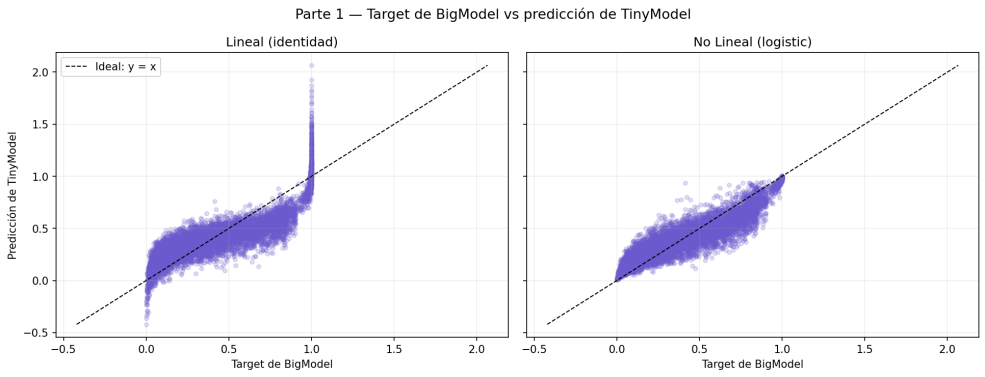
$$z = \left[\frac{42{,}07 - 109{,}07}{157{,}74}\right] = [-0{,}425]$$

Entrenamiento

EJ1 — Parámetros de la Parte 1

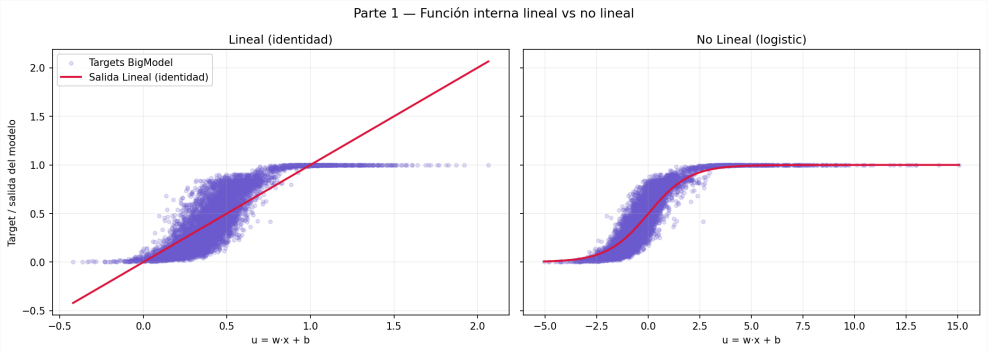
Parámetro	Valor
Modelos comparados	lineal (identity) vs no lineal (logistic)
Activaciones	identity, logistic ($\beta = 1{,}0$)
Optimización	GD
Learning rate	10^{-3}
Batch size	32
Épocas	500
Normalización	z-score
Semillas	$0, 1, 2, 3, 4$

EJ1 — Target vs predicción



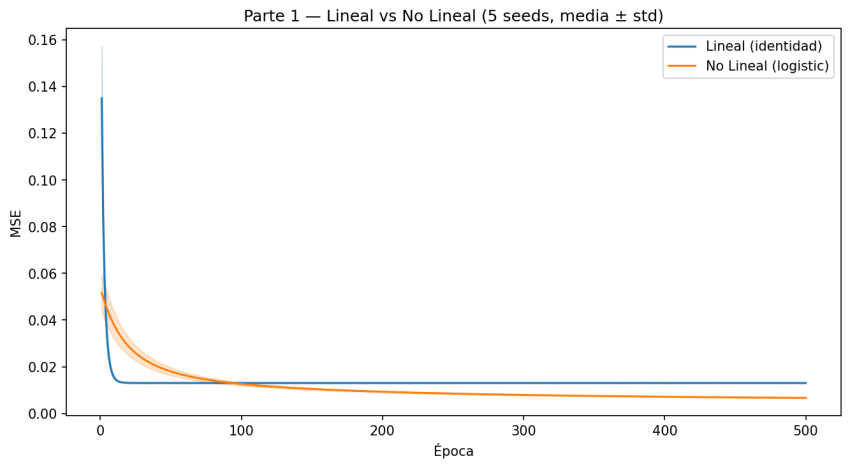
Se comparan las activaciones `identity` y `logistic`.

EJ1 — Función interna



Se comparan las funciones de activación `identity` y `logistic`. El perceptrón no lineal mantiene la salida acotada en $[0, 1]$.

EJ1 — Error MSE



El perceptrón no lineal reduce el **MSE en 48.9 %** respecto del lineal.

EJ1 — Conclusiones Parte 1

- El perceptrón **lineal** muestra **underfitting**: no logra aproximar bien la salida de BigModel.
- El modelo lineal **satura su capacidad** rápidamente y converge a un error más alto.
- El perceptrón **no lineal** captura mejor la relación entrada-salida y mantiene la predicción en el rango $[0, 1]$.
- En este experimento, el no lineal logra un **MSE significativamente menor** sin mostrar una saturación comparable.
- Para generalización, se selecciona el **perceptrón simple no lineal**.

Generalización

EJ1 — Setup experimental de generalización

Parámetro	Valor
Manejo de datos	5-Fold
Modelo	perceptrón simple no lineal
Activación	logística
Optimizador base	GD
Batch size base	32
Learning rate base	10^{-3}
Épocas máximas	500
Early stopping	sí
Semillas	$0, 1, 2, 3, 4$

A partir de esta configuración base, se exploran **batch size**, **learning rate** y **optimizador**.

EJ1 — Métricas de evaluación

Como criterio de diseño, queremos **minimizar los falsos negativos (FN)**: casos donde una operación fraudulenta se clasifica como no fraudulenta. Aun así, también contemplamos el resto de los errores.

- **F2**

$$F_\beta = (1+\beta^2)\cdot\frac{\text{Precision}\,\text{Recall}}{\beta^2\,\text{Precision}+\text{Recall}} \qquad \Rightarrow \qquad F_2 = 5\cdot\frac{\text{Precision}\,\text{Recall}}{4\,\text{Precision}+\text{Recall}}$$

- **MSE**

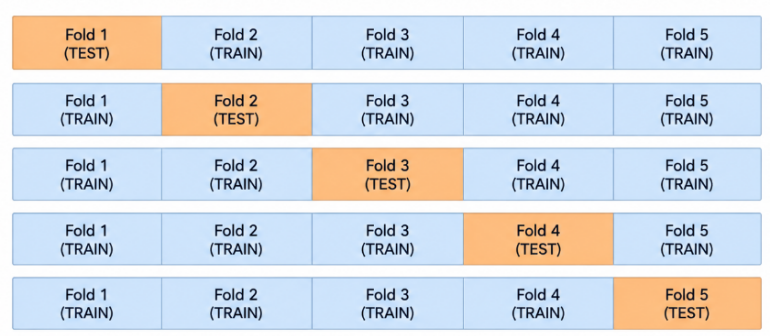
$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2$$

  donde y_i es la salida de **BigModel** y $\hat{y}_i$ es la predicción de nuestro modelo.

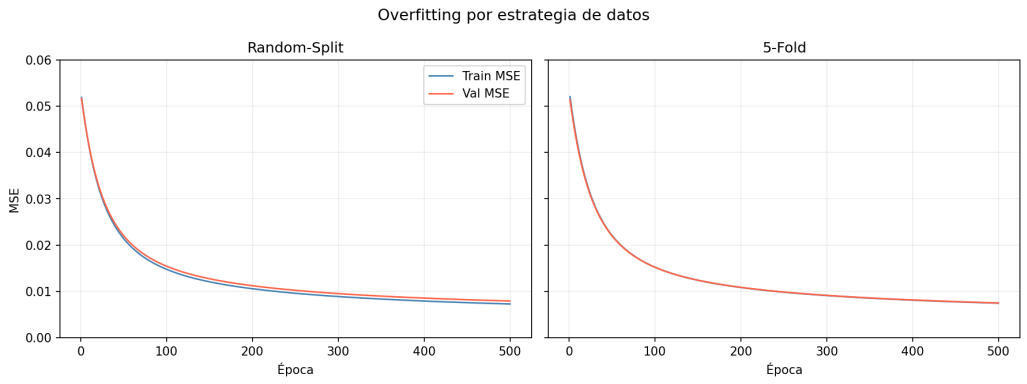
- **Costo** $= 2\,FN + FP$

EJ1 — Estrategias de datos

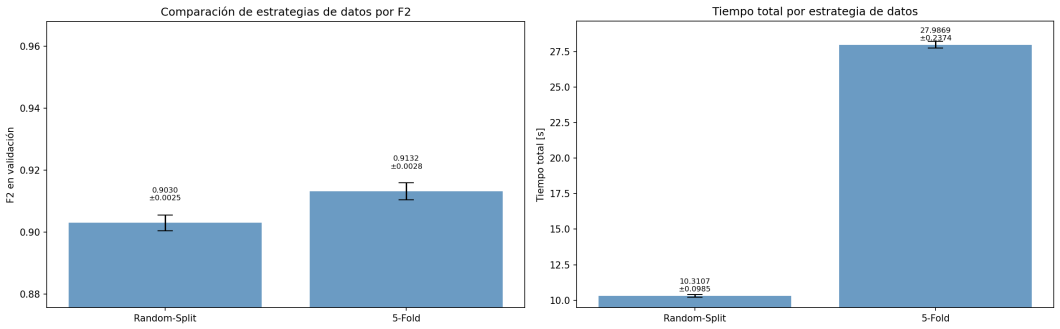
- **Random-Split**: partición aleatoria train/validación.
- **5-Fold**: divide el dataset en folds y rota cuál queda para validación.



EJ1 — Curvas train/val

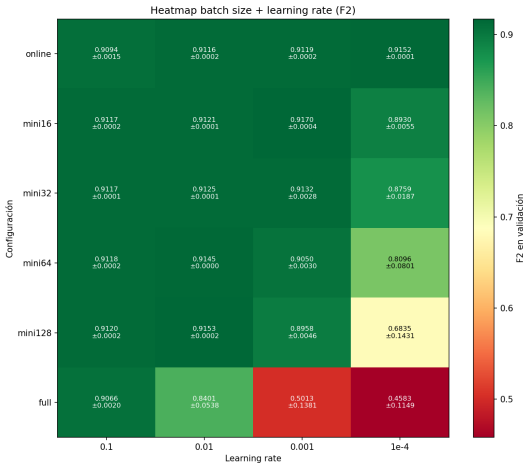


EJ1 — Estrategias de datos

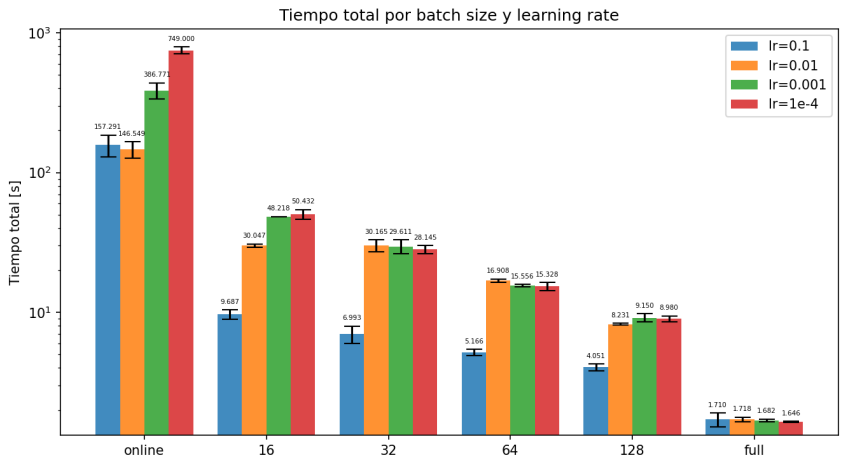


Se selecciona **S3**.

EJ1 — Batch size y learning rate

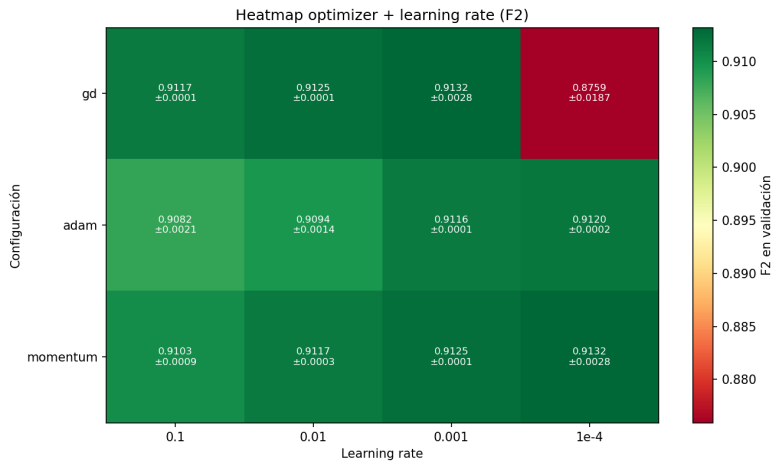


EJ1 — Tiempo por batch size y learning rate

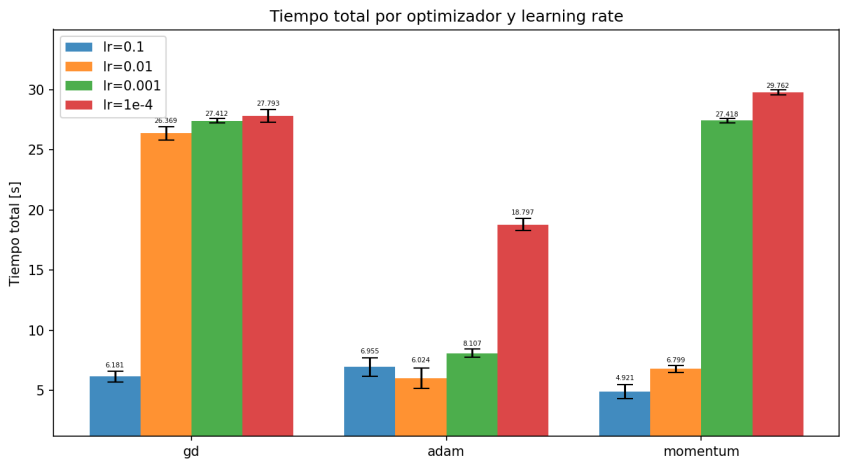


Se elige **batch_size = 16**.

EJ1 — Screening de optimizadores

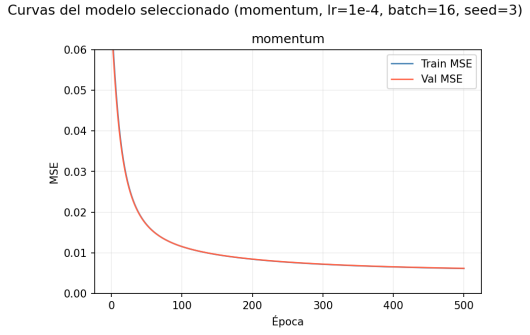


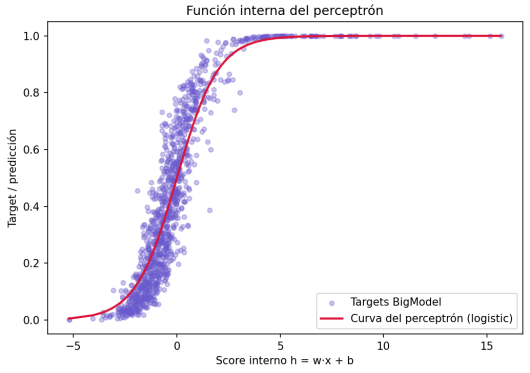
EJ1 — Tiempo por optimizador



Se elige **Momentum** ($\alpha = 0{,}9$).

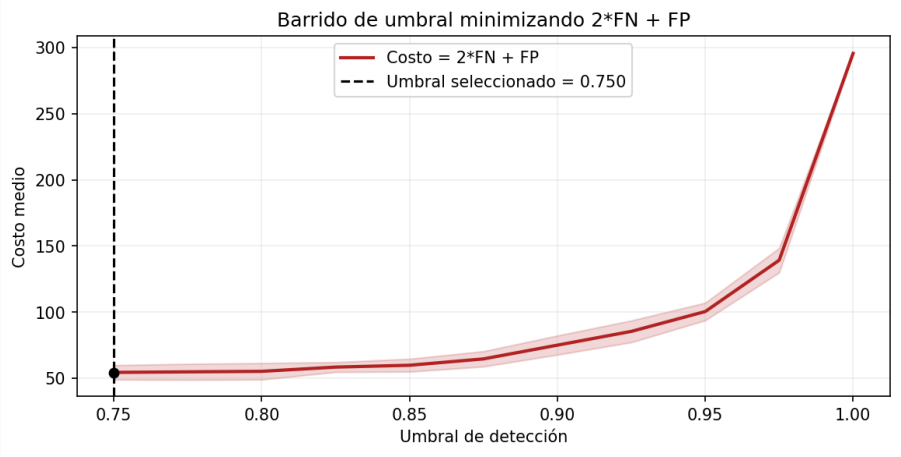
EJ1 — Modelo seleccionado: aprendizaje e interpretación





Modelo: **Momentum ($\alpha = 0{,}9$), lr $= 10^{-4}$, batch = 16** y **S3**.

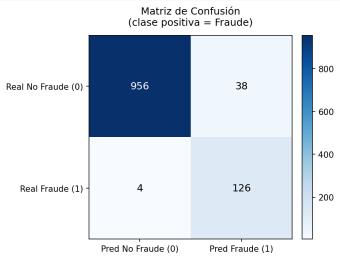
EJ1 — Selección de umbral



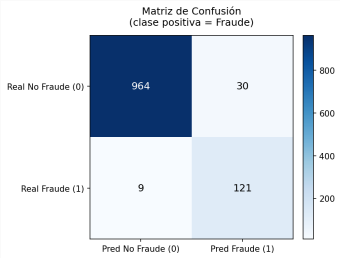
Se selecciona umbral **0.75**.

EJ1 — Matrices de confusión por umbral

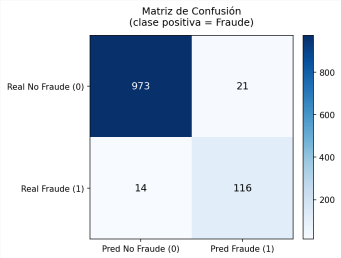
$t = 0{,}75$



$t = 0{,}775$



$t = 0{,}8$



Se selecciona **0.75**, dejando solo **4** casos de fraude real sin detectar.

EJ1 — Modelo final para el cliente

- Estrategia de datos: **S3 (5-Fold CV)**
- Modelo: **perceptrón simple no lineal** con activación **logística**
- Optimizador: **Momentum** ($\alpha = 0{,}9$)
- Learning rate: 10^{-4}
- Batch size: **16**
- Umbral recomendado: **0.75**

- **Precision**: 0,7683
- **Recall**: 0,9692
- **F1**: 0,8571
- **F2**: 0,9211
- **MSE**: 0,00590
- **Costo** ($2FN + FP$): 46

EJ2 — Clasificación de dígitos

Contexto del ejercicio

Problema: CompanyX necesita un sistema automático para clasificar dígitos escritos a mano (0–9) a partir de imágenes de 28×28 pixels.

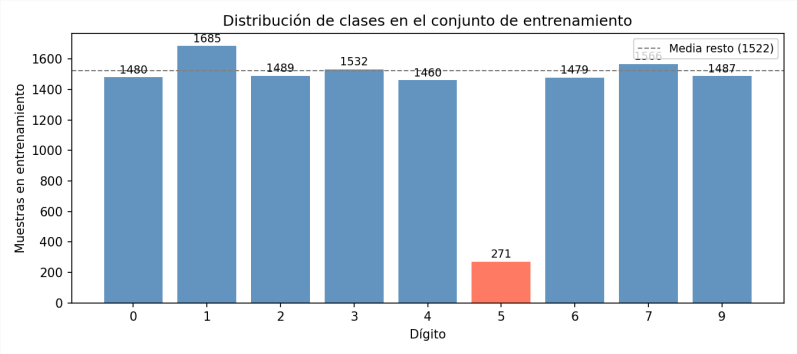
Modelo: Perceptrón Multicapa (MLP) con salida de 10 neuronas (una por dígito), entrenado con MSE.

Estrategia experimental:

1. Ver dependencia entre batch size y learning rate
2. Ver variación de accuracy según optimizador y learning rate
3. Explorar arquitecturas: ¿más capas o más nodos por capa?
4. Evaluar funciones de activación en capas ocultas
5. Analizar el mejor modelo sobre el conjunto de test

División: 80 % train / 20 % val (estratificado) sobre `digits.csv`; `digits_test.csv` como "mundo real".

Análisis del dataset — Distribución de clases



Distribución de muestras por dígito en el conjunto de entrenamiento. Se observa una marcada subrepresentación del 5 y la ausencia total del 8.

Análisis del dataset — Observaciones e hipótesis

Dataset / clase	0	1	2	3	4	5	6	7	8	9
Train (12 449)	1480	1685	1489	1532	1460	**271**	1479	1566	**0**	1487
Test (2 497)	245	283	258	252	245	223	239	257	**243**	252

Observaciones:

- El train **no contiene la clase 8** — el modelo nunca verá un ejemplo de ese dígito.
- La clase 5 tiene solo **271 muestras** ($\sim$2 % del train) contra $\sim$12 % para el resto.
- El test sí incluye 243 muestras del 8 ($\sim$10 % del test).

Hipótesis:

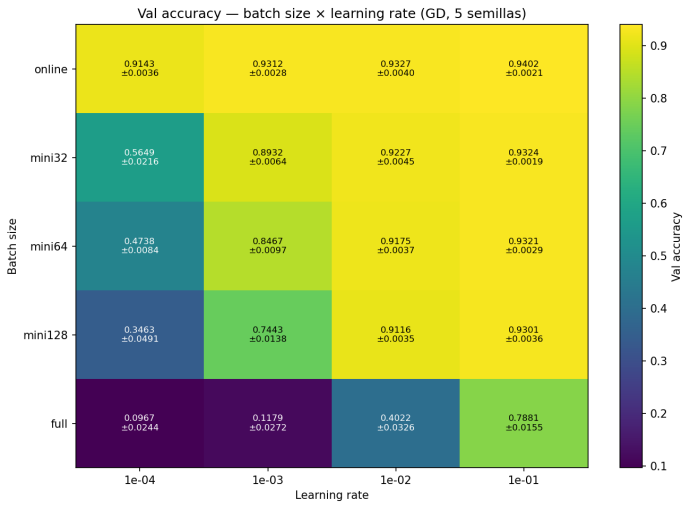
- El 8 será sistemáticamente mal clasificado (techo de test accuracy $\approx$ 90 %).
- El dígito 5 tendrá menor F1 por la escasez de muestras.

Exp1 — Parámetros iniciales

Parámetro	Valor
Arquitectura	[784, 40, 20, 10]
Activación oculta	tanh
Activación de salida	logistic
β (pendiente sigmoidea)	1.0
Inicialización	random_normal, init_scale=0.1
Optimizador	GD (gradiente descendente)
Épocas máximas	500
Función de pérdida	MSE
Early stopping	patience=50, $\varepsilon = 0{,}001$
Seeds	[0, 1, 2, 3, 4]

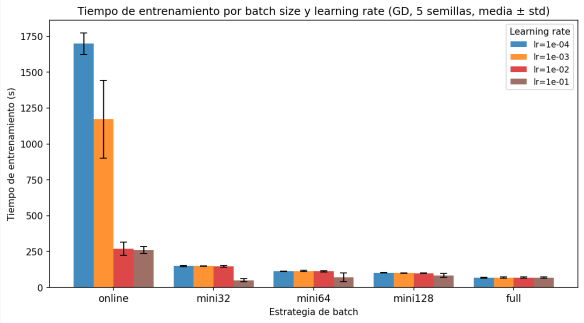
Valores por defecto; se varían batch size y learning rate en este experimento.

Exp1 — Batch size × learning rate



5 batch sizes × 5 learning rates × 5 seeds. La zona óptima se concentra en lr $\in [10^{-3}, 5{\cdot}10^{-3}]$ con batch $\in [8, 128]$.

Exp1 — ¿Online o mini-batch?



Hallazgos:

- El entrenamiento **online** (batch=1) logra la mejor val accuracy, pero es $\sim 10\times$ más lento.
- Mini-batch 32 y 64 obtienen resultados comparables con mucho menor tiempo de entrenamiento.
- El learning rate adecuado **depende del batch size**: batch grande necesita lr más alto; online necesita lr más bajo.

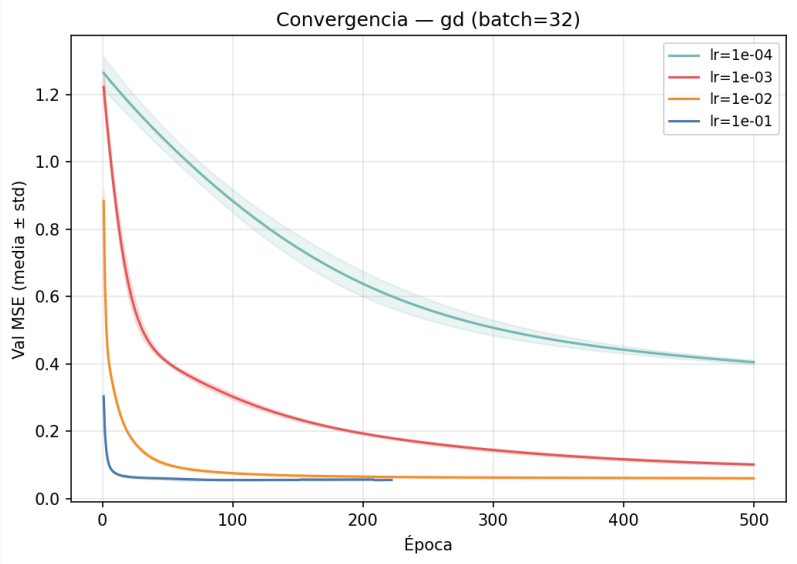
Decisión: batch size = **32**. Buen balance entre velocidad y accuracy; además permite flexibilidad en la elección de lr.

Exp2 — Parámetros iniciales

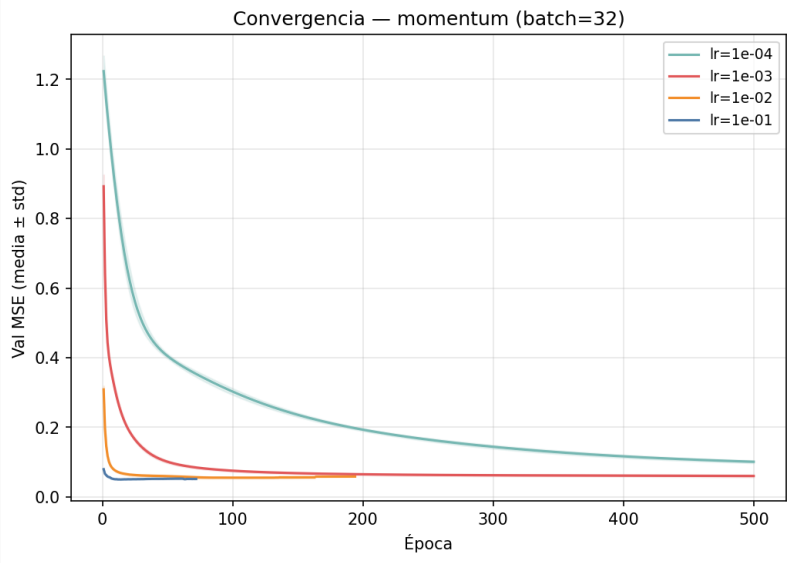
Parámetro	Valor
Arquitectura	[784, 40, 20, 10]
Activación oculta	tanh
Activación de salida	logistic
β (pendiente sigmoidea)	1.0
Inicialización	random_normal, init_scale=0.1
Batch size	32
Épocas máximas	500
Función de pérdida	MSE
Early stopping	patience=50, $\varepsilon = 0{,}001$
Seeds	[0, 1, 2, 3, 4]

Se varían optimizador y learning rate.

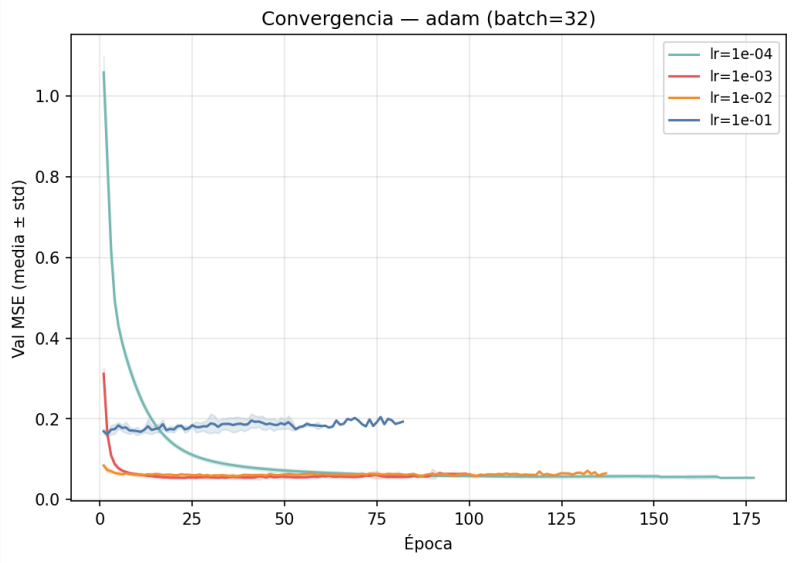
Exp2 — Convergencia: GD



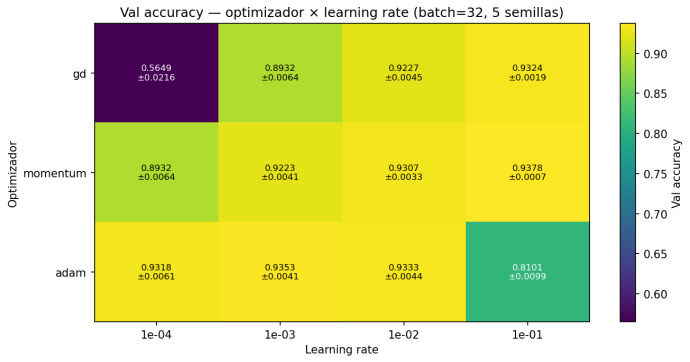
Exp2 — Convergencia: Momentum ($\alpha = 0{,}9$)



Exp2 — Convergencia: Adam (β_1=0,9, β_2=0,999, ε=10^{-8})



Exp2 — Cada optimizador tiene su learning rate óptimo



3 optimizadores $\times$ 4 learning rates $\times$ 5 seeds. GD necesita lr alto ($\sim$0.1), Momentum funciona en rango intermedio, Adam rinde mejor con lr $\sim 10^{-3}$.

Exp2 — ¿Por qué Adam con lr $= 10^{-3}$?

- **Adam** adapta la tasa de aprendizaje por parámetro (momentos de primer y segundo orden), por lo que trabaja bien con lr bajos y converge rápido.
- **GD** no tiene adaptación: necesita lr alto para moverse, pero eso genera inestabilidad.
- **Momentum** queda en un punto intermedio: acelera en direcciones consistentes pero no adapta por parámetro.
- Conclusión: **Adam con lr** $= 10^{-3}$ como optimizador base para los experimentos siguientes.

Exp3 — ¿Más capas o más nodos por capa?

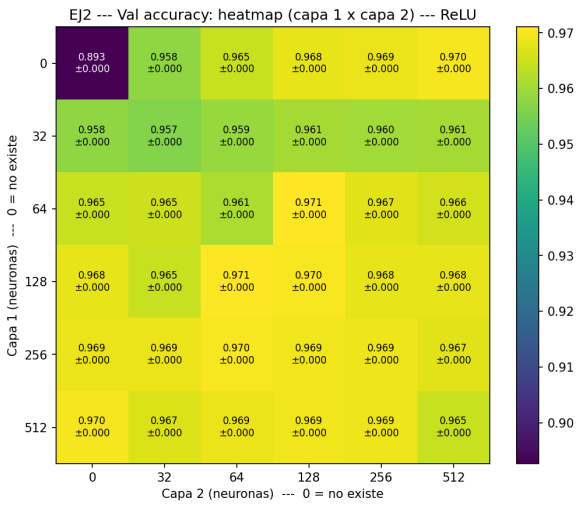
Pregunta: dada una arquitectura de hasta 2 capas ocultas, ¿cómo conviene distribuir los nodos entre la primera y la segunda capa? ¿Conviene una sola capa ancha o repartir en dos capas?

Diseño: barrido 2D de $n_1 \times n_2$ con $n_1, n_2 \in \{0, 8, 16, 32, 64, 128, 256, 512\}$. Convención: $n = 0$ significa que esa capa *no existe* (los pares $(0, n)$ y $(n, 0)$ son el mismo MLP $[784, n, 10]$, así que se computan una sola vez).

- Total: **57 configuraciones únicas** $\times$ 5 seeds = 285 runs.
- Tres clases de arquitectura cubiertas en una sola grilla: *regresión logística* (sin capas ocultas), *1 capa oculta* de tamaño variable, y *2 capas ocultas* con todas las combinaciones.

Configuración: Adam lr= 10^{-3}, batch=32, tanh, patience=50, max_epochs=500, init scale=0.1.

Exp3 — Heatmap de val accuracy ($n_1 \times n_2$, ReLU)

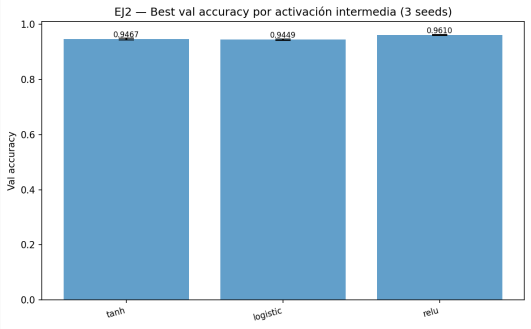


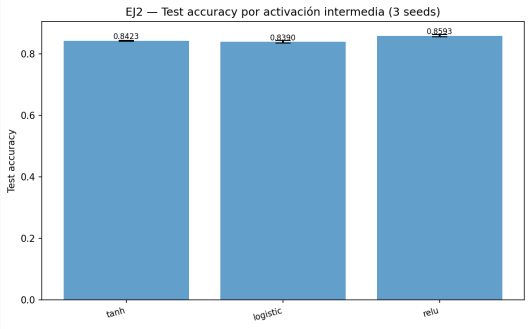
5 seeds por celda. Cada cell: mean $\pm$ std. $n = 0$ significa que esa capa no existe; los pares $(0, n)$ y $(n, 0)$ son el mismo MLP $[784, n, 10]$ y se computan una sola vez.

Exp3 — Lectura del heatmap

- Selección de hiperparámetros realizada sobre **val_acc** (no test) para evitar *test set leakage*.
- La zona top del barrido está prácticamente empatada en val (diferencias de pocos milipuntos): la fila n_1=0 y la columna n_2=0 junto a las celdas grandes muestran que con ReLU una sola capa oculta de 256–512 neuronas alcanza el techo.
- **Wider domina sobre deeper**: agregar una segunda capa NO mejora consistentemente. La primera capa es la más crítica porque procesa los 784 píxeles crudos.
- **Decisión final**: para el modelo del equipo se eligió $[784, 512, 10]$ + ReLU (1 sola capa oculta) — está dentro del empate técnico de los top y es la arquitectura más simple, menos propensa a overfit.
- El test se evalúa una sola vez al final sobre el modelo ya elegido (ver Exp5).

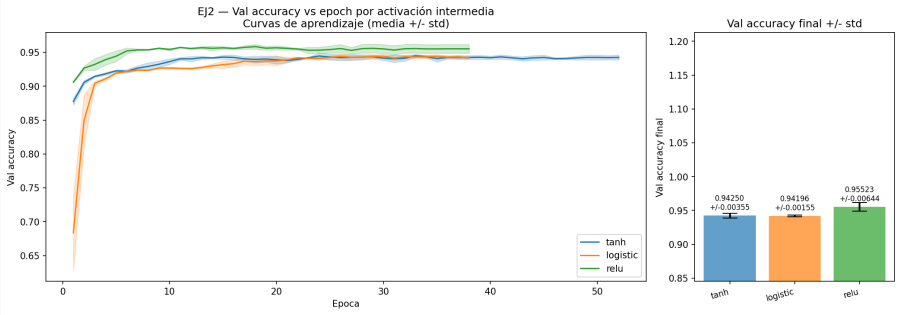
Exp4 — Activaciones en capas ocultas





Arquitecturas [784, 256, 10] y [784, 128, 64, 10], Adam lr= 10^{-3}, batch=32. Tres funciones de activación evaluadas: **logistic**, **tanh** y **ReLU**.

Exp4 — Convergencia por activación



ReLU converge más rápido y alcanza la mejor val accuracy. Logistic sufre de saturación (derivada pequeña) y converge más lento.

Exp4 — Conclusión: elección de activación

Activación	Arquitectura	Val accuracy	Test accuracy
ReLU	[784, 128, 64, 10]	$0{,}9616 \pm 0{,}003$	$0{,}8640 \pm 0{,}002$
Tanh	[784, 128, 64, 10]	$0{,}9497 \pm 0{,}003$	$0{,}8515 \pm 0{,}002$
Logistic	[784, 128, 64, 10]	$0{,}9485 \pm 0{,}004$	$0{,}8467 \pm 0{,}002$

- Las tres funciones de activación dan resultados aceptables.
- **ReLU** es superior: no satura para valores positivos y propaga gradientes más eficazmente.
- **Logistic** es la más lenta en convergencia (derivada pequeña en saturación).
- Decisión: usamos **ReLU** como activación por defecto para los experimentos siguientes.

Exp5 — Configuración del mejor modelo

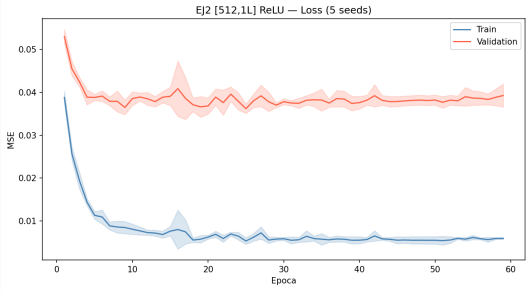
- **Arquitectura**: **[784, 512, 10]** (1 capa oculta, 407 050 parámetros)

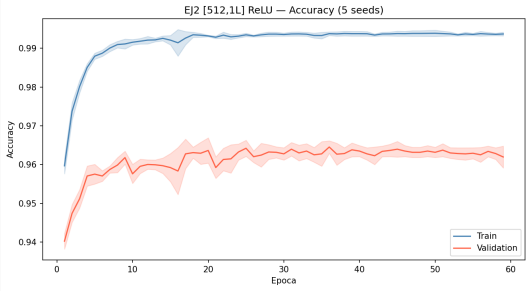
  Ganadora del heatmap ReLU del Exp3.
- **Optimizador**: Adam ($\eta = 10^{-3}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$)
- **Batch size**: 32, init He normal ($\sigma = \sqrt{2/n_{\text{in}}}$)
- **Activación oculta**: **ReLU** (ganadora del Exp4)
- 5 seeds, hasta 150 epochs con early stopping (patience=20)

Resultados (5 seeds, mean $\pm$ std):

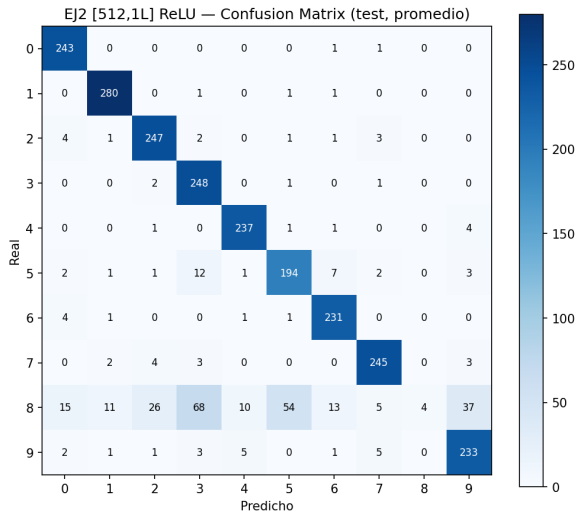
- Val accuracy: $0{,}9661 \pm 0{,}0020$
- Test accuracy: $0{,}8659 \pm 0{,}0046$
- Macro F1: $0{,}8215 \pm 0{,}0083$

Exp5 — Curvas de aprendizaje (5 seeds)



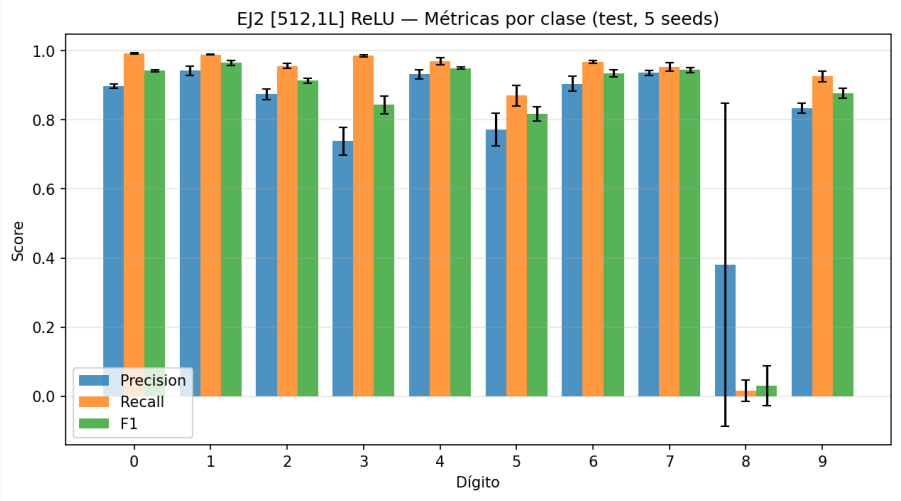


Exp5 — Matriz de confusión (test, promedio 5 seeds)



La fila del 8 es toda incorrecta: el modelo nunca vio un 8. Se confunde principalmente con 5 (23 %) y 3 (22 %).

Exp5 — Métricas por clase (test)



F1 del dígito 8 = 0.000 (ausente en train). El dígito 5 (271 muestras) también tiene F1 bajo ($\sim$0.75). Demás dígitos: F1 $\geq$ 0.84.

Exp5 — Análisis de confusiones por dígito (test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.992	→6: 0.4 %	→7: 0.4 %
1	0.989	→3: 0.4 %	→5: 0.4 %
2	0.954	→0: 1.5 %	→7: 1.2 %
3	0.984	→2: 0.8 %	→5: 0.4 %
4	0.971	→9: 1.6 %	→2: 0.4 %
5	**0.870**	**→3: 5.4 %**	**→6: 3.1 %**
6	0.971	→0: 1.7 %	→1: 0.4 %
7	0.953	→2: 1.6 %	→3: 1.2 %
8	**0.016**	**→3: 28.0 %**	**→5: 22.2 %**
9	0.928	→4: 2.0 %	→7: 2.0 %

El 8, ausente del train, se reparte entre 3 (28 %) y 5 (22 %) — el modelo "los manda a cualquier lado". El 5, con solo 271 muestras en train, sigue siendo la clase peor aprendida.

EJ3 — Mejor accuracy con `more_digits.csv`

Setup del EJ3

Cambio principal: el train pasa a ser `more_digits.csv` ($\sim$15.7k muestras, **10 clases incluyendo el 8**).

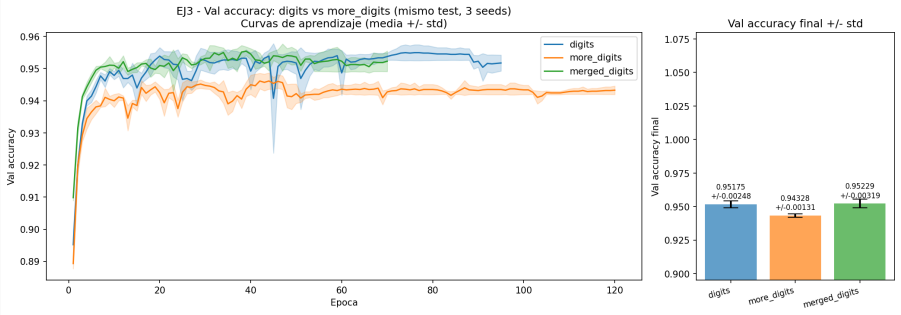
Test: el mismo `digits_test.csv` del EJ2 (mundo real) $\Rightarrow$ comparación directa con EJ2.

Objetivo: accuracy $\geq 98\,\%$ en test.

Técnicas adoptadas:

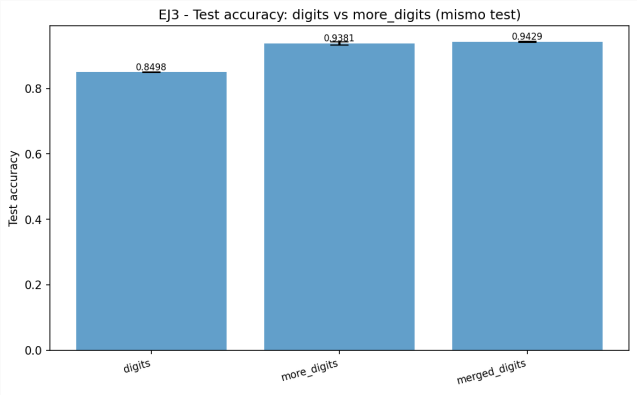
- Arquitectura más grande: $[784, 256, 128, 10]$ (235,146 params).
- Más epochs (150) + patience (20).
- Weight decay liviano ($\lambda = 10^{-5}$) — regularización L2 (Clase 13 - Regularización).
- Init scale más chico (0.05) para mejor estabilidad.
- Batch size 64 (más grande $\Rightarrow$ menos varianza por step).

Comparación de datasets (*aislando el efecto de los datos*)



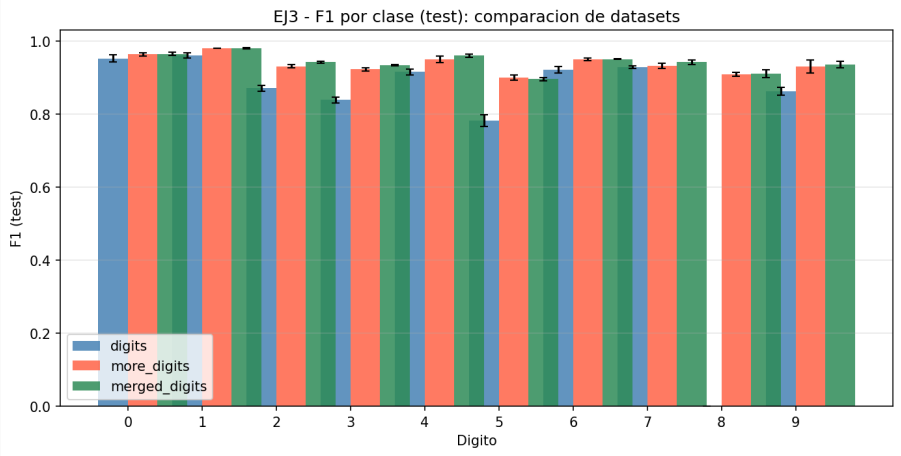
Misma arquitectura $[784, 128, 64, 10]$, mismo test, sólo cambia el train.

Test accuracy: digits vs more_digits



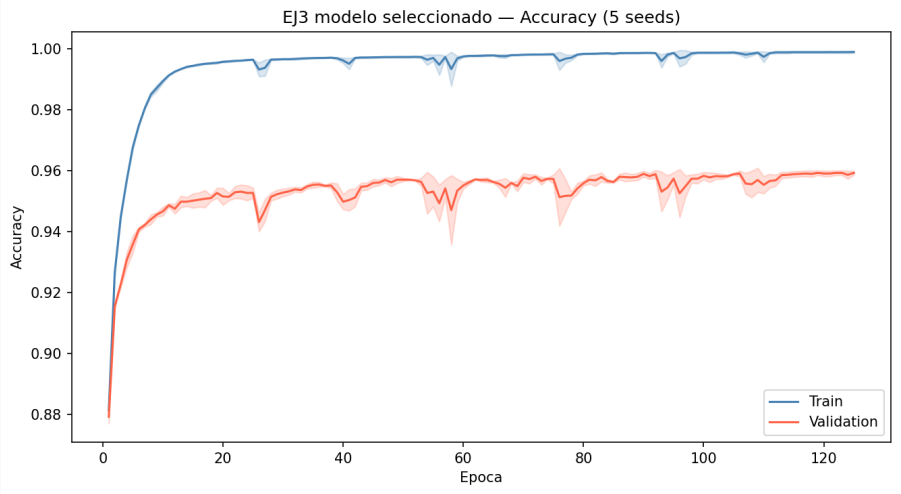
Salto de $\sim$80 % a $>$95 % sólo por incluir la clase 8 en train y tener $\sim$26 % más datos.

F1 por clase (test): digits vs more_digits

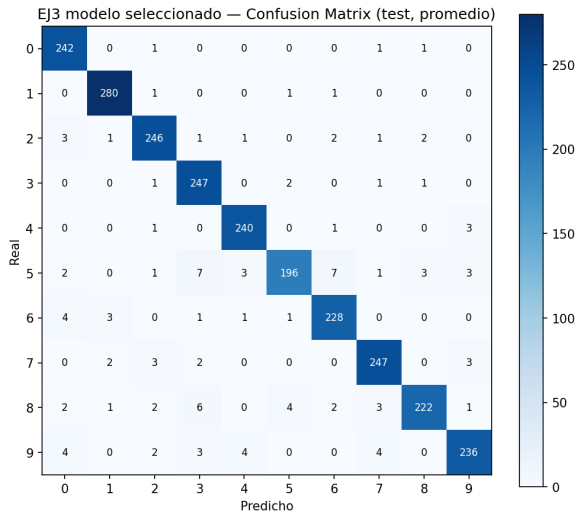


El dígito 8 pasa de F1=0 (no se entrena) a $\sim$0.95.

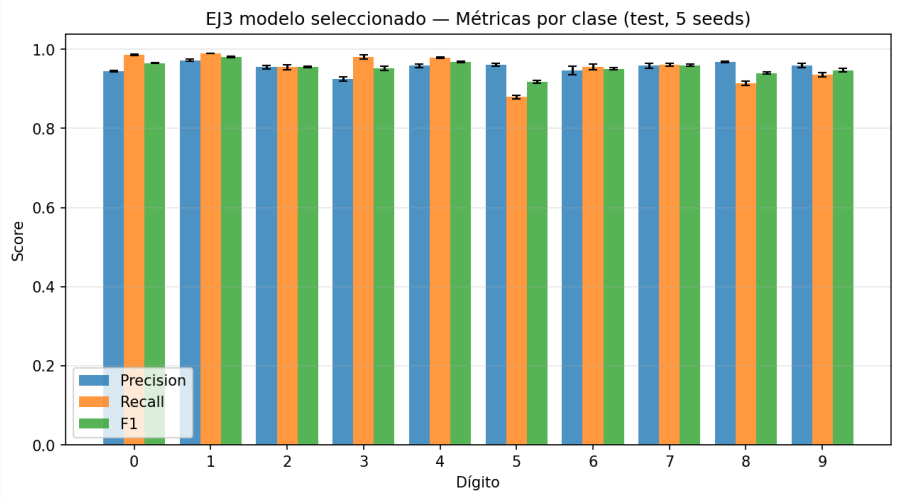
EJ3 modelo seleccionado — accuracy curves



EJ3 — Confusion matrix (test, promedio sobre 5 seeds)



EJ3 — Métricas por clase



EJ3 — Análisis de confusiones por dígito (test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.988	→2: 0.4 %	→7: 0.4 %
1	0.989	→2: 0.4 %	→5: 0.4 %
2	0.957	→0: 1.2 %	→6: 0.8 %
3	0.980	→5: 0.8 %	→2: 0.4 %
4	0.980	→9: 1.2 %	→2: 0.4 %
5	0.879	→3: 3.1 %	→6: 3.1 %
6	0.958	→0: 1.7 %	→1: 1.3 %
7	0.961	→2: 1.2 %	→9: 1.2 %
8	**0.914**	**→3: 2.5 %**	**→5: 1.6 %**
9	0.933	→0: 1.6 %	→4: 1.6 %

El 8 pasa de acc=0.000 (EJ2) a 0.914 (EJ3) — confirma el efecto de incluirlo en el train. Las confusiones residuales del 5 y 8 son con 3, 5, 6 — patrón consistente con similitud visual.

EJ3 — Resultados finales y discusión del 98 %

Mejor resultado obtenido (3 seeds):

- Test accuracy: $0{,}9546 \pm 0{,}0007$ (95.5 %)
- Val accuracy: $0{,}9603 \pm 0{,}0001$
- Macro F1: 0,9536 (vs 0,7942 en EJ2)
- F1 mínimo (dígito 5): 0,918 (vs 0,749 en EJ2)

No alcanzamos 98 % en test. Lo que obtuvimos:

- Train acc $\rightarrow$ 0.999 con val acc plateau en $\sim$0.96 (overfitting clásico, Clase 13 - Regularización).
- El gap train$-$val es de $\sim$0.04, indicando que la red está saturando en su capacidad de generalización con este pipeline.

Para llegar a 98 % se necesitarían: data augmentation (rotaciones, traslaciones — Clase 13), arquitecturas convolucionales, o ensembling. Quedó fuera del scope de un MLP básico.

Cómo llegamos al 98 % — overview

El baseline EJ3 (arquitectura grande $[784, 256, 128, 10]$ + tanh) llega a $\sim$95.5 %. Para superar el 98 % lo que faltaba era:

1. **Activación ReLU** (en vez de tanh) — convergencia más rápida y sin saturación.
2. **Arquitectura simple**: $[784, 512, 10]$ (1 sola capa oculta), consistente con el ganador del heatmap ReLU del Exp3.
3. **Online augmentation**: por cada época y cada batch, aplicar transformaciones aleatorias a las imágenes (rotación, traslación). El modelo nunca ve dos veces el mismo input exacto.
4. **Weight decay** ($\lambda = 10^{-4}$) y **learning rate más conservador** (Adam $\eta = 2{\cdot}10^{-4}$) para evitar overfitting con un train chico.

Sin necesidad de: feature engineering manual (HOG, momentos de Hu, etc.), regularización por noise, dropout o ensemble. Demuestra que la elección correcta de arquitectura + augmentation puede llevar a un MLP simple a 98 %.

Online augmentation — la pieza clave

Idea: en cada época y cada batch, transformar las imágenes del train con perturbaciones aleatorias *antes* de pasarlas al modelo. El modelo "ve" versiones distintas del mismo dígito en cada paso de entrenamiento.

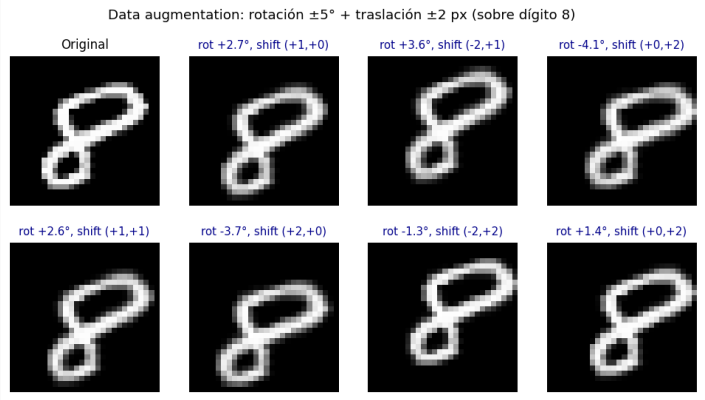
Transformaciones que usamos:

- **Rotación** aleatoria en un rango pequeño.
- **Traslación** aleatoria de pocos píxeles.

Por qué funciona:

- Reduce el overfitting: el dataset efectivo es muchísimo más grande.
- Hace al modelo *invariante* a esos cambios pequeños.
- Ayuda especialmente a clases minoritarias (5, 8): cada época ven nuevas variantes de las mismas pocas muestras.

Augmentation — ejemplo visual sobre un 8



Cada imagen es una rotación + traslación aleatoria del mismo 8 original. La red ve algo distinto cada batch.

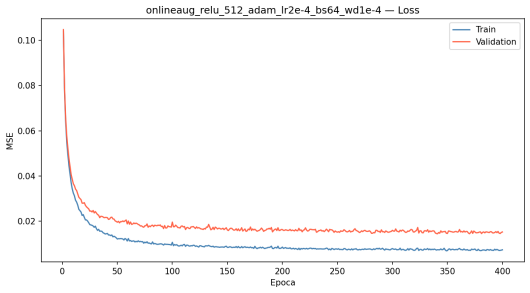
Modelo final — configuración

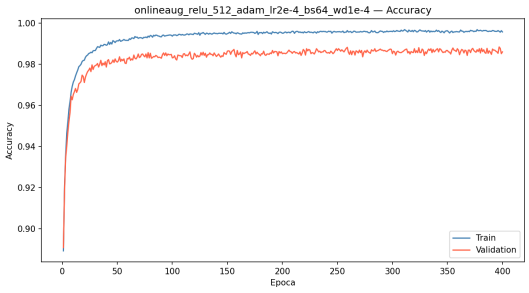
- **Arquitectura**: `[784, 512, 10]` (1 capa oculta, $\sim$407k parámetros).
- **Activación oculta**: **ReLU** + init He. **Salida**: sigmoide.
- **Optimizador**: Adam con $\eta = 2{\cdot}10^{-4}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$.
- **Batch size**: 64.
- **Regularización**: weight decay $\lambda = 10^{-4}$.
- **Online augmentation**: rotación + traslación aleatorias en cada batch, en cada época.
- **Train**: `more_digits.csv` (15.7k muestras, las 10 clases incluyendo el 8).
- **Test**: `digits_test.csv` (mismo del EJ2).

Resultado evaluado sobre el test set:

- **Test accuracy**: 0,9840
- **Macro F1**: 0,9839

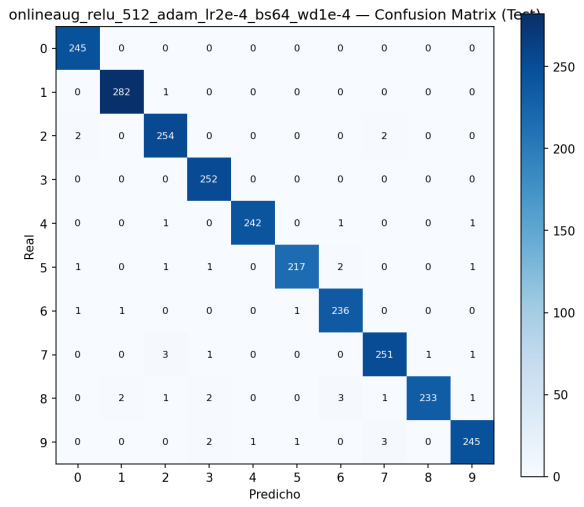
Modelo final — curvas de aprendizaje





El modelo converge suave y estable. La augmentation evita que train_acc dispare por encima del val_acc (control natural del overfitting).

Modelo final — matriz de confusión (test)



Diagonal dominante en todas las clases. El dígito 8 (que era 0 en EJ2) ahora se clasifica correctamente.

Modelo final — per-class F1 (test)

Clase	F1 baseline EJ2	F1 final EJ3
0	0.94	**0.992**
1	0.95	**0.993**
2	0.89	**0.979**
3	0.84	**0.988**
4	0.89	**0.992**
5	0.75	**0.982**
6	0.91	**0.981**
7	0.92	**0.977**
8	**0.00**	**0.977**
9	0.92	**0.978**

- El **8** pasa de F1=0 (imposible en EJ2, ausente del train) a 0,977.
- El **5** (clase minoritaria) sube de 0,75 a 0,982.
- Todas las clases con F1 $\geq$ 0,977 — modelo equilibrado.

Conclusiones

Conclusiones EJ2

- **Exp1 (batch $\times$ lr)**: el learning rate óptimo depende del batch size. Online logra la mejor accuracy pero es 10$\times$ más lento. Se elige batch=32 como compromiso eficiencia/rendimiento.
- **Exp2 (optimizadores)**: cada optimizador tiene su rango óptimo de lr. Adam con lr=10^{-3} domina en accuracy y velocidad.
- **Exp3 (arquitecturas)**: heatmap 2D ($n_1 \times n_2$). *Wider domina sobre deeper* en este rango: la primera capa es la más crítica (procesa los 784 píxeles crudos), la segunda aporta poco. Ganador: $[784, 512, 512, 10]$.
- **Exp4 (activación)**: ReLU, tanh y logistic dan resultados aceptables. ReLU es preferida por convergencia más rápida y sin saturación en valores positivos.
- **Exp5 (mejor modelo)**: **[784, 512, 10]** (1 capa oculta) + ReLU + Adam lr=10^{-3}, batch=32. Ganador del heatmap ReLU.
- **Techo de test accuracy $\approx$ 90 %** por la ausencia del dígito 8 en train. El 8 se confunde con 5 y 3; el dígito 5 (271 muestras) también sufre.

Referencias al material

- Clase 9 — Optimización (GD, SGD, Momentum, Adam)
- Clase 11 — Perceptrón Multicapa (forward pass, T. Aprox. Universal)
- Clase 12 — Métricas y Sobreajuste (accuracy, F1, confusion matrix)
- Clase 13 — Backprop (regla de la cadena, δ por capa)
- Clase 13 — Regularización (early stopping, L2 weight decay)